

Recursion and backtracking

CSC103 Programming Fundamentals / Lecture 19

Dr. Muhammad Farhan

Associate Professor of Computer Science
Department of Computer Science
COMSATS University Islamabad
Sahiwal Campus

Fall 2026

The problem for this lecture

Trace factorial(4) with base case factorial(0)=1. Then write the recursive method for inputs 0..12 using int.

Reject n outside the supported range before computing.

Learning objectives

- Identify base and recursive cases
- Trace recursive stack frames
- Explain recursive search and backtracking

CLO-2

Base case and progress

Recursive call stack

Recursion and iteration

Backtracking search

Base case and progress

- A recursive method calls itself directly or indirectly.
- A base case returns without another recursive call.
- Every permitted input must progress toward a base case.

Example

```
sumTo(0) = 0  
sumTo(n) = n + sumTo(n-1), n > 0
```

Recursive call stack

- Each recursive call keeps its own parameters and pending work.
- Returns unwind in reverse order.
- The deepest base case determines the first returned result.

Example

```
sumTo(3): waits for 3 + sumTo(2)
sumTo(2): waits for 2 + sumTo(1)
sumTo(1): waits for 1 + sumTo(0)
sumTo(0): returns 0
```

Recursion and iteration

- Both can express repetition.
- A loop can often use constant auxiliary space for a simple sum.
- Recursive depth consumes stack space and can cause `StackOverflowError`.

Example

Iterative sum:

```
int total = 0;  
for (int i=1; i<=n; i++) total += i;
```

Backtracking search

- Backtracking explores a choice, then explores alternatives when needed.
- A base case recognises a complete candidate.
- Immutable partial results naturally preserve the earlier choice state.

Example

```
binary("", 2)
  binary("0", 1)
    outputs 00, 01
  binary("1", 1)
    outputs 10, 11
```

A recursive definition needs a domain

- $\text{factorial}(0)=1$ supplies the base case.
- For positive n , $\text{factorial}(n)=n*\text{factorial}(n-1)$.
- For this `int` implementation, restrict n to $0..12$ so that results fit the chosen type.

Example

```
factorial(4) = 4 * 3 * 2 * 1 = 24  
13! exceeds the positive int range.
```

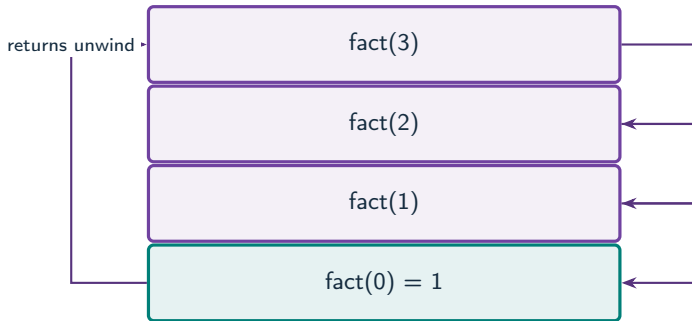

Returning is different from making a call

- A factorial frame waits for the smaller factorial before multiplying.
- The base case provides the first completed answer.
- Unwinding performs the pending multiplications from the deepest frame upward.

Example

```
fact(0) returns 1
fact(1) returns 1 * 1
fact(2) returns 2 * 1
fact(3) returns 3 * 2
```

Recursive calls unwind in reverse order



What to notice

Calls move toward the base case; returned values then flow back to waiting callers.

Key distinctions

Technique	Stored work	Good fit
Iteration	Loop state and accumulator	Simple repeated accumulation
Direct recursion	Pending work in frames	A problem with a smaller same-form case
Backtracking	Choices and search state	Exploring alternative candidates

These distinctions determine which operation or control structure fits the problem.

First example: methods

```
static int sumTo(int n) {  
    if (n < 0) throw new IllegalArgumentException("negative n");  
    if (n == 0) return 0;  
    return n + sumTo(n - 1);  
}  
  
static void binary(String prefix, int remaining) {  
    if (remaining == 0) { System.out.println(prefix); return; }  
    binary(prefix + "0", remaining - 1);  
    binary(prefix + "1", remaining - 1);  
}
```

First example: execution

```
System.out.println(sumTo(4));  
binary("", 2);
```

First example: result

10

00

01

10

11

Each binary choice reduces the remaining depth by one.

Recursion and iteration

Backtracking search

Guided practice

Trace $\text{factorial}(4)$ with base case $\text{factorial}(0)=1$. Then write the recursive method for inputs 0..12 using int.

Attempt the solution before continuing.
Use the definitions and examples from this lecture.

Solution strategy

- Reject n outside the supported range before computing.
- Return 1 at $n==0$.
- For a positive n , return n multiplied by $\text{factorial}(n-1)$.

The next program uses fixed inputs so its result can be reproduced.

Complete solution

```
public class Solution19 {  
    public static void main(String[] args) throws Exception {  
        System.out.println(factorial(4));  
        System.out.println(factorial(0));  
    }  
    static int factorial(int n) {  
        if (n < 0 || n > 12) {  
            throw new IllegalArgumentException("n must be in 0..12");  
        }  
        if (n == 0) return 1;  
        return n * factorial(n - 1);  
    }  
}
```

Execution trace

Returning frame	Calculation	Returned value
factorial(0)	Base case	1
factorial(1)	$1 * 1$	1
factorial(2)	$2 * 1$	2
factorial(3)	$3 * 2$	6
factorial(4)	$4 * 6$	24

Follow the changing state in execution order.

Solution result

24

1

Why this result follows

For a positive n , return n multiplied by $\text{factorial}(n-1)$.

A common incorrect approach

```
static int f(int n) { return n + f(n-1); }
```

Example

There is no base **case**. Add an appropriate base **case** and enforce a domain that progresses toward it.

Boundary and failure checks

Check $n=0$, $n=1$ and $n=4$. Then verify rejection for $n=-1$ and $n=13$.

Pending multiplications: $4*\text{fact}(3)$, $3*\text{fact}(2)$, $2*\text{fact}(1)$, $1*\text{fact}(0)$.
Returns: 1,1,2,6,24. Larger values can overflow int.

Check your understanding

Why does backtracking need to restore mutable state?
What causes the direct recursive sum to terminate?

Write a prediction and explain the rule that supports it.

Answer and explanation

A later branch must start from the state before the earlier choice. `sumTo` reduces nonnegative `n` until `n==0`.

There is no base case. Add an appropriate base case and enforce a domain that progresses toward it.

Compare cases and predict outcomes

Returning from	Calculation	Returned value
fact(1)	$1 * 1$	1
fact(2)	$2 * 1$	2
fact(3)	$3 * 2$	6

Discuss

Explain each expected result before running the example. Which case would reveal a wrong assumption?

Apply the idea

Independent practice

Write recursive `sumTo(n)` for 0 through 100. Trace `sumTo(3)`, including the base-case return.

1. Work through the input by hand.
2. Write or correct the program.
3. Justify the expected result.

Solution and reasoning

```
static int sumTo(int n) {  
    if (n < 0 || n > 100)  
        throw new IllegalArgumentException("n outside 0..100");  
    if (n == 0) return 0;  
    return n + sumTo(n - 1);  
}
```

- The calls are sumTo(3), sumTo(2), sumTo(1), sumTo(0).
- The returned values are 0, 1, 3 and 6.
- The stated bound limits recursion depth and keeps these sums within int.

Lecture summary

- base and recursive cases
- recursive stack frames
- recursive search and backtracking
- Reject n outside the supported range before computing.
- Return 1 at $n==0$.
- For a positive n , return n multiplied by $\text{factorial}(n-1)$.

After-class practice

Generate all three-character binary strings recursively.
Explain why there are eight results.

Syllabus reading

Deitel Ch 18

Next lecture: One-dimensional arrays